

Ejercicio 5 – Android - Layouts (3)

En esta actividad vas a crear un nuevo proyecto y continuarás conociendo diferentes tipos de Layout desde el modo de “Diseño” de Android Studio.

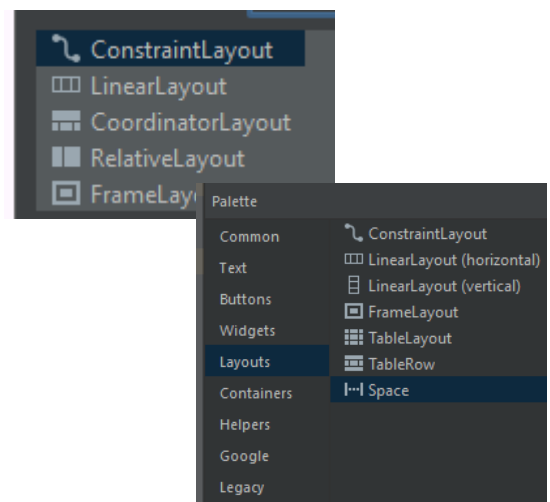
Primero crea un nuevo proyecto en Android Studio del tipo “**Empty Views Activity**”.

Llámale “**Layouts3**”, el Lenguaje seleccionado “Kotlin” y aunque tenemos diversas plataformas y APIs en las que utilizar nuestra aplicación, nosotros nos centraremos en aplicaciones para teléfonos y tablets, en cuyo caso tan sólo tendremos que seleccionar la API mínima (es decir, la versión mínima de Android) que soportará la aplicación. Android 9.0. como versión mínima (API 28):

Una vez configurado todo pulsamos el botón **Finish** y Android Studio creará por nosotros toda la estructura del proyecto y los elementos indispensables que debe contener.

Los **view groups** más populares son:

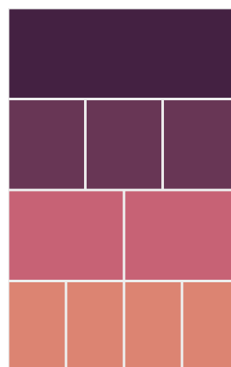
- **FrameLayout** (Actividad 3.1)
- **LinearLayout** (Actividad 3.1)
- **RelativeLayout** (Actividad 3.2)
- **ConstraintLayout** (Actividad 3.2)
- **TableLayout (TableRow)**
- **GridLayout**
- **CoordinatorLayout**



1. Aplicación de Layouts en Android Studio

a. TABLELAYOUT

el TableLayout organiza views en filas y columnas de forma tabular:



Para crear las filas se usa el componente **TableRow** dentro del **TableLayout**. Cada celda es declarada como un view de cualquier tipo (imagen, texto, otro group view, etc.) dentro de la fila. Sin embargo, puedes crear una celda con otro tipo de view. Esto hará que todo el espacio de la fila sea ocupado por el objeto.

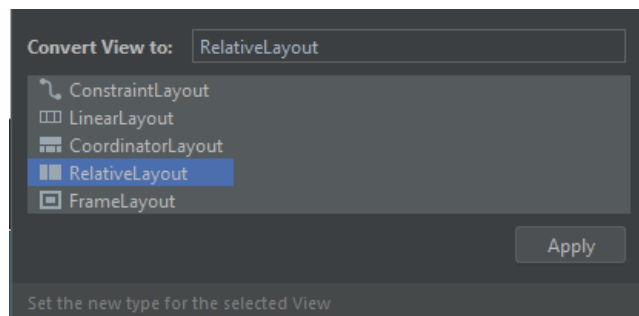
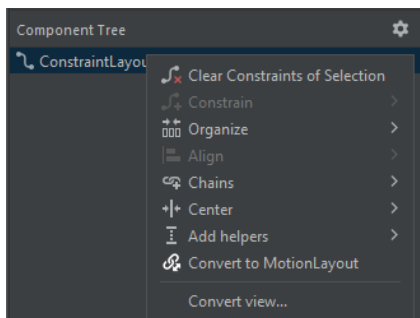
El **TableRow** trae consigo un parámetro llamado **android:layout_column** para asignar la columna a la que pertenece cada celda en su interior. Incluso puedes usar el parámetro **weight** para declarar pesos de las celdas.

El **ancho de cada columna** es definido tomando como referencia la celda más ancha. Pero también podemos definir el comportamiento del ancho de las celdas con los siguientes atributos:

- **android:shrinkColumns:** Reduce el ancho de la columna seleccionada hasta ajustar la fila al tamaño del padre.
- **android:stretchColumns:** Permite rellenar el espacio vacío que queda en el **TableLayout**, expandiendo la columna seleccionada

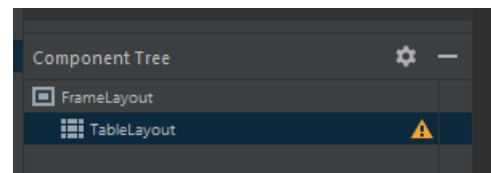
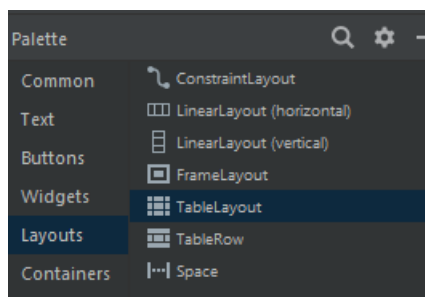
Ahora tu parte práctica en Android Studio:

- En el Layout xml que te ha creado por defecto, cámbiale el nombre y llámale "table_layout".
- En el modo de **diseño** del layout anterior, tendríamos que cambiar el tipo de Layout en "Component Tree" a **"TableLayout"**:



Pero como verás, no aparece. Para utilizarlo tenemos 2 opciones:

- Crear el **TableLayout** dentro de uno de los otros tipos de Layouts que hemos visto en Actividades anteriores: Seleccionaríamos uno de ellos y posteriormente en la ventana de "Palette" seleccionaremos "Layouts>TableLayout":



- O desde la ventana de Código XML, crearemos un **TableLayout** desde 0:

```
activity_main.xml x MainActivity.kt x
1 <?xml version="1.0" encoding="utf-8"?>
2 <TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:padding="16dp"
6     android:stretchColumns="1">
7
8 </TableLayout>
```

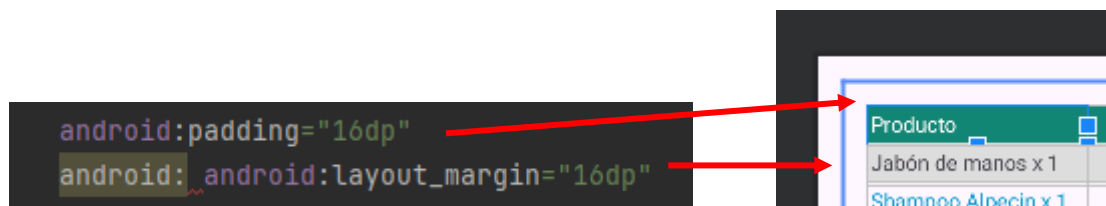
Créalo de esta manera



Con “**stretchColumns**”: Se rellenará el espacio vacío que queda en el TableLayout expandiendo la columna seleccionada (empieza en 0).

Con “**padding**”: Agregamos un margen interior al layout.

Si hubiéramos utilizado “**margin**”, estableceríamos un margen exterior al layout:



- C) Ahora hay que definir todas las filas y columnas que vamos a utilizar. El diseño final de la tabla tiene que ser similar a esta:

Producto	Subtotal
Jabón de manos x 1	2€
Shampoo Alpecin x 1	10€
Pastas Layana x 2	18€
Detergente Tucolor x 1	13,4€
Subtotal	43,4€
Costo envío	10€
Cupón	-5€
Total	48,4€

- D) Para generar esta tabla, la vamos creando **fila a fila** con “**TableRow**”, y a la vez definiendo las columnas con “**android:layout_column**”.

Vete insertándolas a través de un “**TextView**” donde introducirás además el contenido de la tabla. En el caso de la **1ª fila** será:

```
<TableRow android:background="#128675">

    <TextView
        android:layout_column="0"
        android:padding="3dip"
        android:text="Producto"
        android:textColor="@android:color/white" />

    <TextView
        android:layout_column="2"
        android:padding="3dip"
        android:text="Subtotal"
        android:textColor="@android:color/white" />

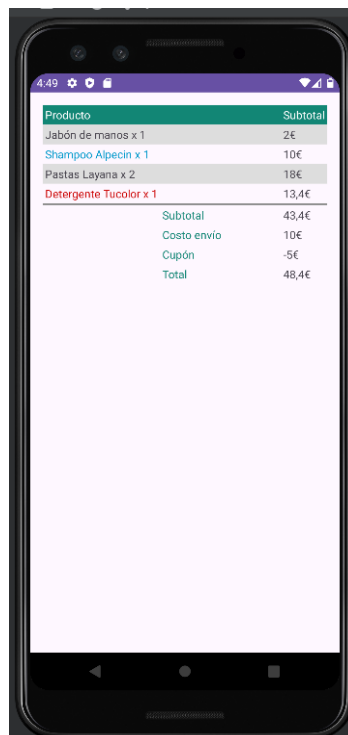
</TableRow>
```

Vete insertando **el resto de filas** atendiendo a los mismo criterios (cambiando columnas, colores, ...)

E) La **línea** que separa los totales defínela como:

```
<View
    android:layout_height="2dip"
    android:background="#FF909090" />
```

F) Una vez conseguido el diseño final, muéstralo en un dispositivo, el resultado debe ser similar a este:

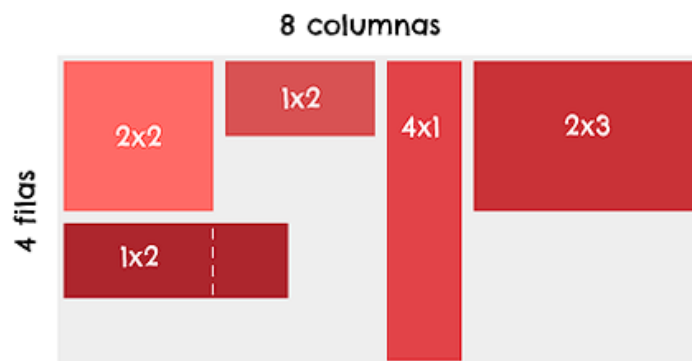


b. GRIDLAYOUT

UN GRIDLAYOUT ES UN VIEWGROUP QUE ALINEA SUS ELEMENTOS HIJOS EN UNA CUADRÍCULA (GRILLA Ó GRID). NACE CON EL FIN DE EVITAR ANIDAR LINEAR LAYOUTS PARA CREAR DISEÑOS COMPLEJOS.

Además, otra diferencia es que con TableLayout, las filas y las columnas se añaden dinámicamente a medida que construye la tabla y con GridLayout, los tamaños de fila y columna se definen en la definición del diseño.

Veamos un ejemplo:



Cuadrícula de 8x4 con 5 views,

Su funcionamiento se basa en un sistema de índices con **inicio en cero**. Es decir, la primera columna (o fila) tiene asignado el índice 0, la segunda el 1, la tercera el 2, etc.

Los atributos más importantes del **GridLayout** son:

- **columnCount:** Cantidad de columnas que tendrá la grilla.
- **rowCount:** Cantidad de filas de la cuadrícula.
- **useDefaultMargins:** Si asignas el valor de true para establecer márgenes predeterminadas entre los ítems.

En cuanto a sus parámetros, es posible especificar la cantidad de filas y columnas que ocupará una celda a través de los atributos:

- **android:layout_rowSpan**
- **android:layout_columnSpan**.

Esta característica nos posibilita para **crear diseños irregulares** que un TableLayout no permitiría.

Atendiendo a la imagen con el Grid del ejemplo de arriba:

- Es posible **expandir** las celdas de forma horizontal y vertical. Incluso es posible proyectar views de forma cruzada. Por ejemplo, el elemento que se encuentra en la segunda fila con las especificaciones 1x2 se cruza en la columna 3. Esto se debe a que su ancho es de 3 unidades, pero su atributo **columnSpan** es igual a 2, lo que facilita al framework crear el cruce si existe espacio en blanco.
- También puedes especificar a qué **fila y columna** pertenece cada **view** con los atributos:

- `android:layout_row`
- `android:layout_column`.

Ejemplo:

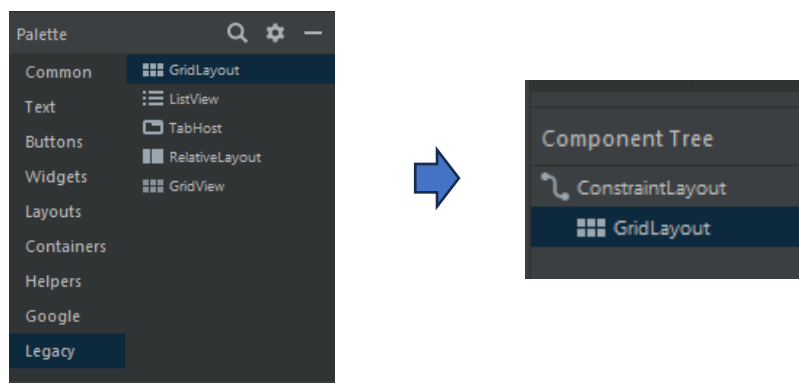
```
<TextView
    android:id="@+id/celda_1"
    android:layout_column="0"
    android:layout_row="0"
    android:text="Celda 1" />
```

Este TextView se encuentra en la **posición (0,0)**

AHORA

TU PARTE PRÁCTICA EN ANDROID STUDIO:

- A) Crea un nuevo Layout y llámale "grid_layout.xml"
 - B) Ahora tenemos que introducir un GridLayout dentro de la ventana de diseño. Para ello también tenemos 2 maneras:
- 1) Sobre el "ConstrainLayout" que tenemos creado por defecto, añadiremos un GridLayout que encontramos en el grupo "Legacy>GridLayout" de la ventana "Palette":

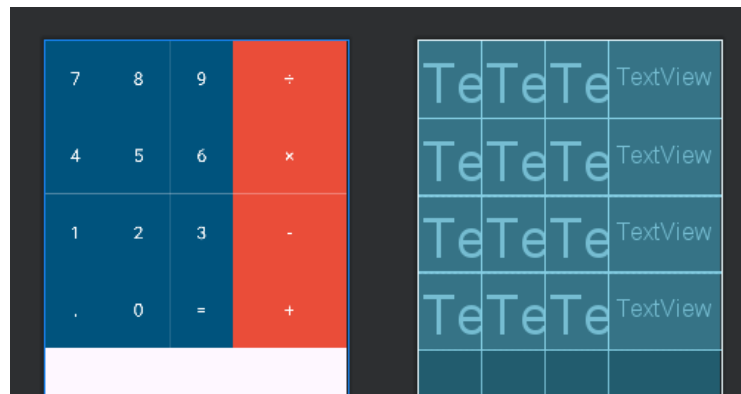


- 2) Lo definimos por código xml:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:alignmentMode="alignBounds"
6      android:columnCount="4"
7      android:rowCount="4">
8
9  </GridLayout>
```

Hazlo de esta manera para definir ya la **matriz de 4 filas x 4 columnas**.

- C) Ahora vas a realizar el diseño de una calculadora similar a este:



Cuenta con una tabla de 4x4 con la última columna de un tamaño diferente. Además vamos a definir 2 estilos diferentes (azul y rojo) para los 2 tipos de botones que se visualizan,

D) Los **botones azules**, vete definiéndolos de una manera similar a esta:

```
<TextView
    android:id="@+id/numero_7"
    style="@style/AppTheme.BotonCalculadora.Azul"
    android:layout_column="0"
    android:layout_row="0"
    android:layout_gravity="fill_horizontal"
    android:text="7" />
```

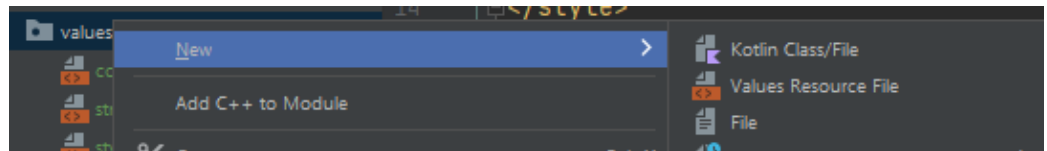
- Como hemos visto, con “**layout_colum**” y “**layout_row**” definirás su posición
- Incluye los **identificadores (id)** siguiendo el mismo patrón.
- Incluye la sentencia “**stylo**” que veremos después.
- Con **layout_gravity**:
 - **fill_horizontal** se utiliza para llenar todo el recipiente horizontalmente.
 - **fill_vertical** se utiliza para llenar todo el recipiente verticalmente.
 - **fill** se utiliza para llenar todo el recipiente tanto horizontal como verticalmente.

E) Diseña **los botones rojos** siguiendo este patrón:

```
<TextView
    android:id="@+id/textView10"
    style="@style/AppTheme.BotonCalculadora.Rojo"
    android:layout_column="3"
    android:layout_gravity="fill_horizontal"
    android:layout_row="0"
    android:text="÷" />
```

F) Ahora, hay que definir en el archivo de estilos (styles.xml) las características de los botones:

- Dentro de la carpeta **res/values/**, si no lo tienes creado, crea el archivo **"styles.xml"**:



- Ahora introduce el siguiente código:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3
4 <style name="AppTheme.BotonCalculadora" parent="TextAppearance.AppCompat.Headline"
5     <item name="android:textColor">@android:color/white</item>
6     <item name="android:gravity">center</item>
7     <item name="android:padding">36dp</item>
8     <item name="android:layout_width">wrap_content</item>
9     <item name="android:layout_height">wrap_content</item>
10 </style>
11
12 <style name="AppTheme.BotonCalculadora.Azul" parent="AppTheme.BotonCalculadora">
13     <item name="android:background">#00537D</item>
14 </style>
```

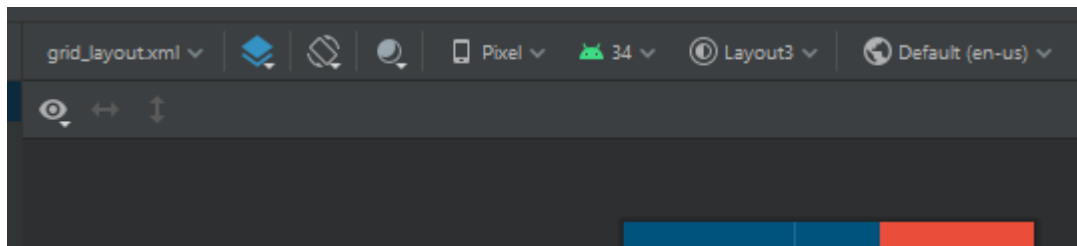
- En el tema "BotonCalculadora" definimos un color para el texto, un alineamiento, un tamaño de celda, y el tipo de "auto-ajuste" o "autocompletado" que utilizaremos.
- Después, se define el estilo del **botón azul**.
- Ahora añade tú el código del **botón rojo**.

G) Una vez acabado, ejecuta la App en un dispositivo (virtual o físico) y comprueba que está todo correcto. Para ello necesitarás cambiar la ejecución de la Activity configurada en el archivo **MainActivity.kt**. Una vez cambiado este archivo al **Layout** que acabas de hacer, ya puedes depurarlo, y enviarlo a los dispositivos para probarlo.

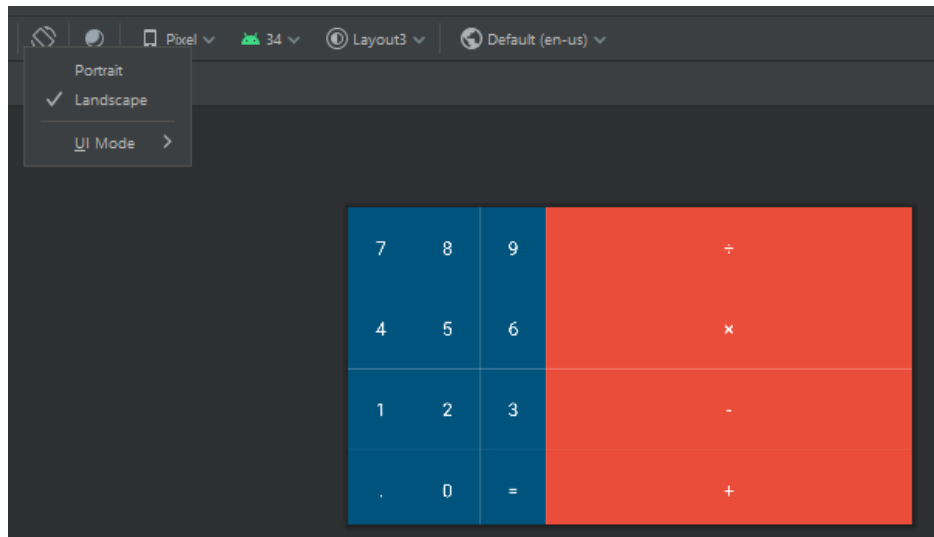
Guarda el proyecto y sube los archivos a Moodle.

NOTA:

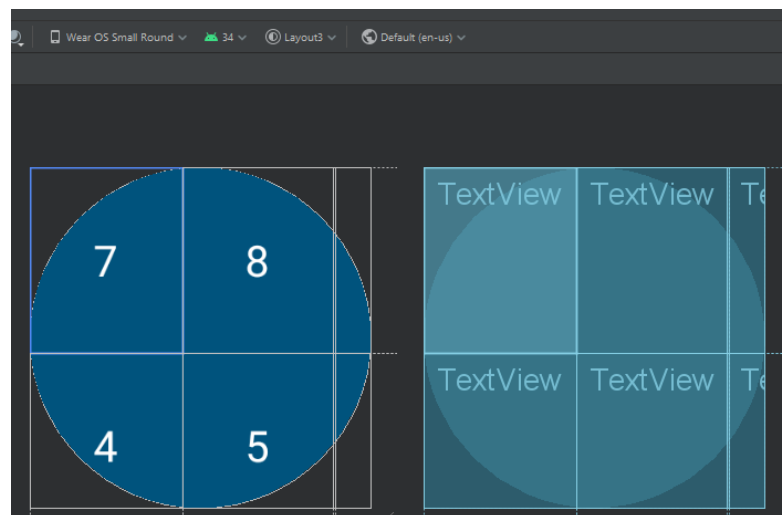
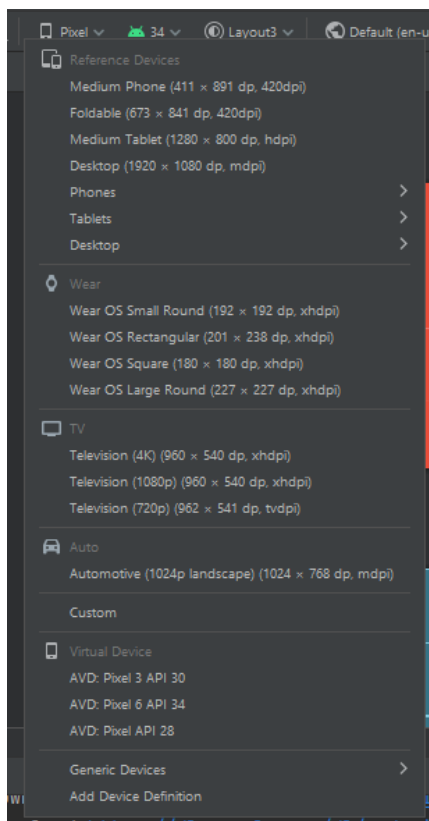
Prueba a utilizar las diversas opciones que nos da la barra de herramientas del lienzo:

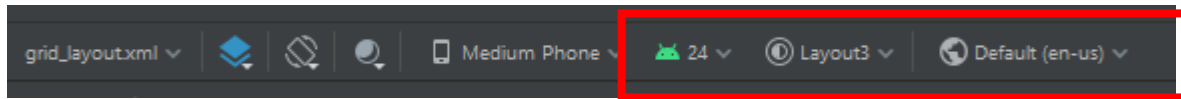


Modo apaisado (o Landscape):



Otros tipos de dispositivos en el Pre-View (antes de lanzarlo al dispositivo virtual):





Otras versiones Android / Diferentes temas / Diferentes idiomas (traducciones) o escrituras.